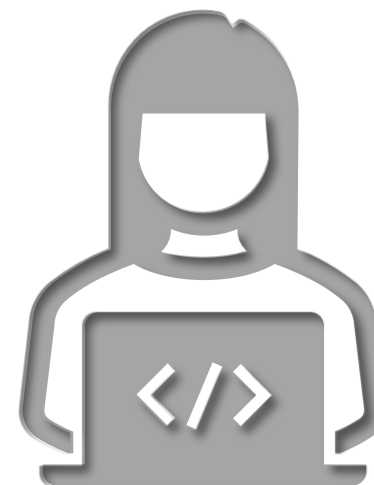
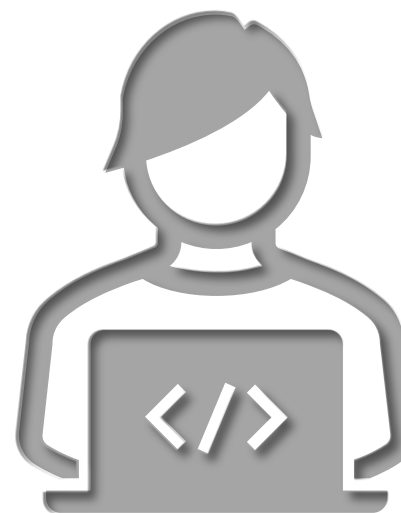
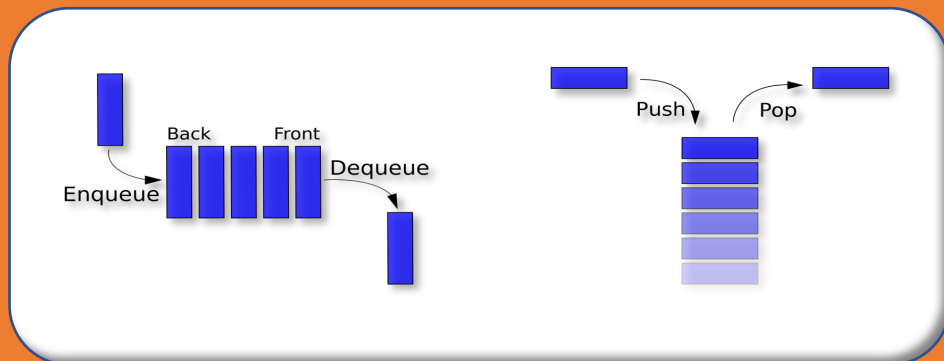


Synthèse :

Types Abstraits de Données

Files & Piles



DÉFINITIONS & CONCEPTS

Les Types Abstrais de Données

DÉFINITIONS

Type Abstrait de Données (ou Type de Données abstrait) :

- Spécification d'un ensemble de traitements (**primitives**) sur un ensemble de données dont le type pourra être **concret** ou **abstrait**. Cette spécification ne précise pas l'implémentation du **T.A.D**, le **T.A.D**. joue alors le rôle d'un cahier des charges où l'on se limitera à décrire :
 - La nature des données à mémoriser ;
 - Les primitives (traitements) qui pourront être appliquées aux données.

Type Concret de Données :

- Synonyme de **structure de données**, c'est-à-dire l'implémentation d'un **T.A.D.**, l'action d'implémenter étant de choisir les types intermédiaires (concrets) qui vont permettre de mémoriser les informations (ex : tableaux, listes, chaîne de caractères, classes...).

DÉFINITIONS

Type simple/de base :

- Type prédéfini permettant de travailler avec des données constituées d'une valeur unique.
- **Exemples** : entier, flottant, booléen et chaîne de caractères

Type structuré :

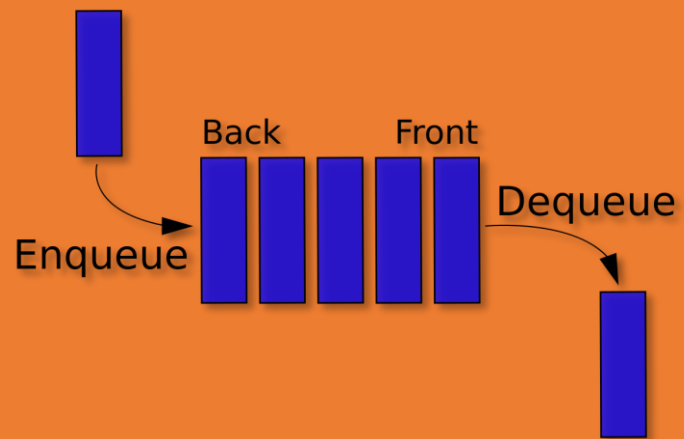
- Type prédéfini permettant de travailler avec des données constituées de plusieurs valeurs. Ces types nécessitent une certaine structuration de l'espace mémoire consommé pour permettre l'accès aux valeurs des données.
- **Exemples** : tableau, liste, dictionnaire, tuple

: Type abstrait :

- Type non natif qui sera construit à partir d'autres types.

LA FILE

DÉFINITION & IMPLÉMENTATION EN PYTHON



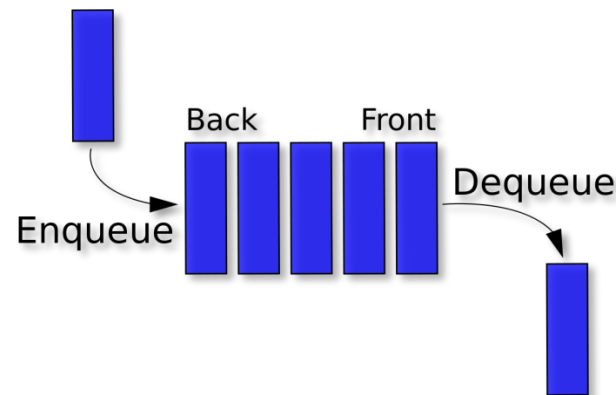
LA FILE

Description



- Description générale de la T.A.D. file :

- Une file est un type de données linéaires où la première donnée entrée dans la file, sera la 1^{ère} à sortir. La 2^e à rentrer, la 2^e à sortir, etc.
- Les données sont traitées comme des personnes dans une file d'attente.
- On parle en anglaise de « **First In, First Out** » (FIFO) ou en français « **Premier Entré, Premier Sorti** » (**PEPS**)



IMPLÉMENTATION EN Impératif



L'implémentation d'une file peut se faire directement avec une **liste** avec le langage Python.

C'est la façon d'ajouter/récupérer une valeur qui va faire que la liste se comporte comme une file.

On a alors deux stratégies :

1. On ajoute les nouvelles données par la gauche et on sort les anciennes par la droite.



2. On ajoute les nouvelles données par la droite et on sort les anciennes par la gauche.



Nous vous proposons un exemple d'implémentation selon les deux stratégies à la page suivante.

IMPLÉMENTATION EN Impératif



Implémentation d'un comportement de file avec Python.

```
file = []
nouvDonnée = "Nouvelle donnée"

# La file :
# -----

# Entrée par la gauche & sortie par la droite =>[..., ...]=> :
file = [nouvDonnée] + file
sortie = file.pop()

# Entrée par la droite & sortie par la gauche <=[..., ...]<= :
file.append(nouvDonnée)
sortie = file.pop(0)
```

IMPLÉMENTATION EN P.O.O.



La programmation orientée objet se prête tout à fait bien pour définir de nouveaux types de données.

Une seule classe est nécessaire pour définir et exploiter une file.

La mémorisation des valeurs est assurée par une liste

Dans la page suivante nous vous proposons une implémentation minimale avec 3 méthodes :

- **enfiler()** : Pour ajouter une nouvelle valeur en queue de file.
- **défiler()** : Pour sortir et récupérer la valeur en tête de file.
- **estVide()** : Retourne VRAI si la file est vide, sinon retourne FAUX.

IMPLÉMENTATION EN P.O.O.



Implémentation d'une classe File en Python :

```
class File:
    """Objet conteneur respectant le principe F.I.F.O./P.E.P.S :
    Premier Rentré, Premier Sorti (First In, First Out)"""

    def __init__(self):
        self.file = [] # Une liste est utilisée comme conteneur de base

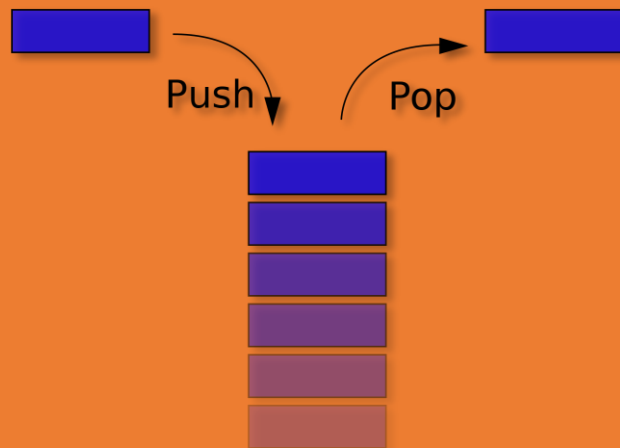
    def enfiler(self, valeur):
        """Rajoute la valeur donnée en paramètre en queue de la file."""
        self.file.append(valeur)

    def défiler(self):
        """Retourne et supprime de la file la valeur située en tête."""
        return self.file.pop(0)

    def estVide(self):
        if len(self.file) == 0:
            return True
        else:
            return False
```

LA PILE

DÉFINITION & IMPLÉMENTATION EN PYTHON



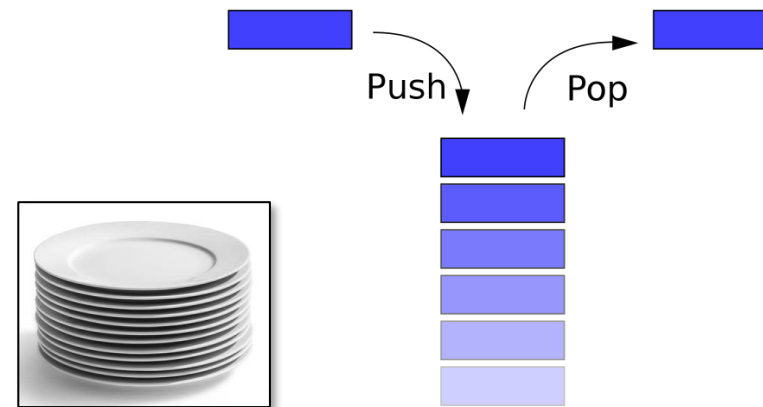
LA PILE

Description



- Description générale de la T.A.D. pile :

- Une file est un type de donnée linéaire où la dernière donnée entrée dans la pile, sera la 1^{ère} à sortir.
- Les données sont traitées comme des assiettes empilées.
- On parle en anglais de « **Last In, First Out** » (LIFO) soit en français « **Dernier Entré, Premier Sorti** ».



IMPLÉMENTATION EN Impératif



L'implémentation d'une pile peut se faire comme pour la file directement avec une **liste**.

C'est encore la façon d'ajouter/récupérer une valeur qui va faire que la liste se comporte comme une pile.

On retrouve encore deux stratégies :

1. On ajoute et sort les données par la gauche.



2. On ajoute et sort les données par la droite.



Nous vous proposons un exemple d'implémentation selon les deux stratégies à la page suivante.

IMPLÉMENTATION EN Impératif



Implémentation d'un comportement de pile avec Python.

```
# La pile :  
# -----  
# Entrée & sortie par la gauche <=> [..., ...] :  
pile = [nouvDonnée] + pile  
sortie = pile.pop(0)  
  
# Entrée & sortie par la droite [..., ...]<=> :  
pile.append(nouvDonnée)  
sortie = pile.pop()
```

IMPLÉMENTATION EN P.O.O.



La programmation orientée objet se prête également bien pour implémenter une pile.

Une seule classe est nécessaire.

La mémorisation des valeurs est assurée par une liste

Dans la page suivante nous vous proposons une implémentation minimale avec 3 méthodes :

- **empiler()** : Pour ajouter une nouvelle valeur en sommet de pile.
- **dépiler()** : Pour sortir et récupérer la valeur en sommet de pile.
- **estVide()** : Retourne VRAI si la pile est vide, sinon retourne FAUX.

IMPLÉMENTATION EN P.O.O.



Implémentation d'une classe Pile en Python :

```
class Pile:
    """Objet conteneur respectant le principe L.I.F.O. :
    Dernier rentré, premier sorti (Last In, First Out)"""

    def __init__(self):
        self.pile = [] # Une liste est utilisée comme conteneur de base

    def empiler(self, val):
        """Ajoute la valeur donnée en paramètre au sommet de la pile."""
        self.pile.append(val)

    def dépiler(self):
        """Retourne et supprime de la pile la valeur située à son sommet."""
        return self.pile.pop()

    def estVide(self):
        if len(self.pile) == 0:
            return True
        else:
            return False
```

Fin de la *synthèse*
